



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E) 2026–27,

Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 5

Team No.: 1

Project Title: Concurrent Web Server for University Resource Portal

Team Members

Roll Number	Student Name	Signature
2520030305	Sura.Rahul Reddy	

1. Abstract

Concurrent Web Server for University Resource Portal is a Linux-based project that demonstrates efficient handling of multiple client requests simultaneously. The server uses a **pool of worker threads** and a shared connection queue to process TCP requests concurrently. **Mutexes and semaphores** are used for synchronization, while **mmap()** provides efficient access to static files. The server also supports **request logging, signal-based control, configuration reload, and resource monitoring**. The project demonstrates practical concepts of **multithreading, synchronization, socket programming, memory mapping, and resource management**.

2. Problem Statement

In a university environment, a large number of students may simultaneously access online academic resources such as notices, course materials, examination schedules, assignments, and other static files. A sequential web server that processes one client request at a time can become a performance bottleneck when many students access the portal simultaneously.

The problem is to design and implement a **concurrent web server** capable of efficiently handling multiple TCP client connections while safely managing shared resources. The server must prevent race conditions and ensure proper synchronization when multiple worker threads access the shared connection queue, logs, and other resources.

This project addresses the problem by implementing a **multithreaded TCP server in C** using a pool of worker threads. Incoming client connections are placed into a shared queue, from which worker threads retrieve and process requests. **Mutexes** provide mutual exclusion for shared data structures, while **semaphores** coordinate the availability of connections in the queue.

The server uses **mmap()** for efficient access to static files and provides requested resources to clients over TCP. It also maintains server activity and request logs and uses signals for server control and configuration reload. The project demonstrates how Operating Systems concepts can be applied to build an efficient and responsive concurrent server.

3. Objectives

1. To design and implement a concurrent TCP web server in C.
2. To accept and handle multiple client connections simultaneously.
3. To implement a pool of worker threads for concurrent request processing.
4. To implement a shared connection queue for managing incoming client requests.
5. To use mutexes and semaphores for synchronization and safe access to shared resources.
6. To serve static HTML, text, and other files to connected clients.
7. To use memory-mapped files using **mmap()** for efficient file access.
8. To implement server activity and request logging for monitoring and debugging.
9. To use signals for server control and configuration reload operations.
10. To monitor active worker threads and server resource usage.
11. To demonstrate Operating Systems concepts such as concurrency, synchronization, thread management, and resource management.

4. Proposed Methodology

The project will be designed and implemented as a **Linux-based C/C++ application** that demonstrates communication and control between processes using signals.

1. Server Initialization: Create a TCP socket, bind it to a port, and listen for clients.
2. Client Handling: Accept incoming client connections and place them in a shared queue.
3. Worker Threads: Create a pool of worker threads to process client requests concurrently.
4. Synchronization: Use mutexes and semaphores to safely manage the shared connection queue.
5. Request Processing: Workers read HTTP requests and identify the requested files.
6. File Serving: Use **mmap()** to efficiently access and serve static files.

7. Logging: Record client requests and server activity in log files.
8. Signal Handling: Use signals for server control and configuration reload.
9. Resource Monitoring: Monitor active threads and server resource usage.
10. Execution Flow: Initialize → Accept Client → Queue Request → Process Request → Serve File → Log → Close Connection.

The methodology provides a practical demonstration of **Concurrent Web Server for University Resource Portal, synchronization, and Linux system calls**.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
TCP Socket – socket(), bind(), listen(), accept()	Used for client-server communication.
POSIX Threads – pthread_create()	Creates worker threads for concurrent request processing.
Mutex – pthread_mutex_lock(), pthread_mutex_unlock()	Protects shared resources from simultaneous access.
Semaphores – sem_wait(), sem_post()	Synchronizes the shared connection queue.
mmap()	Provides efficient memory mapped file access.
Signals – signal(), sigaction()	Used for server control and configuration reload
File I/O – open(), close(), read(), write()	Used to access and serve requested files.
Logging	Records client requests and server activity.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030305	Sura Rahul Reddy	Full Project completion

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the operating environment for developing, compiling, and executing the process communication program.
C / C++	Used to implement process creation, signal handling, communication, and process control.
GCC / G++ Compiler	Compiles the C/C++ source code into an executable program.
POSIX Libraries	Provides APIs and functions for signals, processes, and synchronization.
Linux Terminal / Shell	Used to compile, execute, test, and monitor processes and signals.

8. Expected Outcome

The project is expected to produce a functional concurrent Linux-based web server capable of handling multiple client requests simultaneously.

1. The server will successfully create and initialize a TCP listening socket.
2. Multiple client connections will be accepted and processed concurrently.
3. A pool of worker threads will process client requests efficiently.
4. A shared connection queue will manage incoming client connections.
5. Mutexes and semaphores will ensure safe access to shared resources.
6. The server will serve HTML, text, and other static files to clients.
7. mmap() will provide efficient access to requested files.
8. Server activity and client requests will be recorded in log files.
9. Signals will support server control and configuration reload.
10. The server will monitor active worker threads and resource usage.
11. Testing with multiple clients will demonstrate concurrent request processing.
12. The project will provide practical understanding of multithreading, synchronization, socket programming, memory mapping, signals, logging, and resource monitoring.

9. GitHub Repository

GitHub Repository Link:

<https://github.com/ITZRAHULPLAYZ/KLH-CSE-2025-Team01-ConcurrentWebServer>

Repository Created:

☐ Yes ☐ No

**Faculty Name and
Signature**

Faculty Name: Dr.Harika

Signature:

Remarks:

Faculty Use

Project Approved:

☐ Yes ☐ No

Date:

Faculty Signature: